从文献方法到本题原创方案

Hash Join Spill-to-Disk 动态调度
2026年10月13日 | 在原12页设计说明基础上新增研究与推导

此次新增的五个方案

编号	方案	主要贡献
01	相邻双Probe的联合驻留	利用Miss回调的动作顺序，让当前与下一目标同时驻留，减少对周期B的依赖。
02	BUILD写入守恒与阶段承诺	证明已决定在BUILD落盘的分区，其累计写字节不随落盘时刻改变；重新评估早释放的价值。
03	带暴露时间的淘汰账本	在Hit不可见时，利用自己的淘汰决策、落盘持续时间和读回损失学习。
04	有计算预算的多情景动作规划	评估几个候选动作集合在短预测窗口内的状态演化，捕捉连续换入换出。
05	预取收益证书与等待信息价值	分清确定的当前收益和提前选victim造成的信息损失；给预取划分证据等级。

原创性的准确含义

这里的“原创”表示本次针对题面自行推导、设计并给出验证路径的具体方案。并未完成覆盖所有研究的优先权检索，因此不宣称世界首创或已达到论文发表标准。用到的通用缓存、幽灵历史与短窗口优化思想会明确注明。

实现与实验状态

01、02与05已完成构造性模型检查；03、04仍是设计提案。新增方案没有完整C++实现，没有新的200组性能实测，也不能继承旧实现13.77%的收益数字。

四条已核验的文献路线

文献	原方法	本题迁移边界
R1 / GDS / 1997	把访问局部性、取回成本和对象大小纳入优先级；使用全局老化变量。	可作代价缓存基线。分区BUILD会增长，SPILL总有写费；原算法需要每次访问通知。
R2 / ARC / 2003	通过已淘汰页的历史信息调整近期性与频率之间的偏好。	借鉴反馈思想。原文采用等大小页及逐次请求，本题不能把其复杂度或收益直接套用。
R3 / Saving Private Hash Join / 2025	外部Hash Join与动态多算子内存管理；第6节研究按实际规模调整分配。	借鉴全局分配。原系统可按页管理、压缩和处理流水线，本题固定原子分区与输入大小。
R4 / Predictive Marker / 2018	预测参与决策，预测失效时保持传统在线算法的兜底能力。	借鉴稳健思想。本题对象大小、写成本和观测缺失改变模型，不能沿用其竞争比证明。

对上一版创新表述的校正

“代价/大小”已在R1中有成熟先例；淘汰历史来自R2及更早工作；动态多算子分配在R3中有直接相关研究。因此这些适合作为文献基线或组合设计，不应单独宣称原创。此次新增重点是规则推导、接口利用和观测受限时的具体机制。

核验方式

已打开四篇原始论文；重点阅读R1第4节、R2幽灵缓存及ARC机制、R3第3与第6节、R4第3节。文末提供出版社或作者机构原文链接。

题面列出的其他参考项

未核验到“Spilling-Based Parallelism for Hash Joins, VLDB 2023”的准确对应记录；也未确认题面所述“Cache-Oblivious Hash Joins, CIDR 2022”的对应项。本报告不沿用这些未经核验的书目信息。

先限定信息与合法动作

基础实现能知道什么

三个回调只能看到State快照、当前Miss/INSERT目标以及next_op。Hit_Count和Miss_Count给出全局探测总数，却不给出每个Hit的分区。周期只有INSERT与PROBE计数，SWITCH不触发；不能偷偷读取输入文件的完整未来轨迹。

一个可识别性反例

设A、B始终驻留，B=1000，在没有周期回调的200次Hit区间内，历史H1为100次A后100次B，H2为(A,B)交替100次。两者可产生相同大小、驻留状态、计数和最后分区；其局部访问模式却不同。标准接口无法从该快照唯一还原历史。

原创方案的接口适配

方案	原接口可行性	信息要求
01	可行，需核对官方接口对多LOAD列表的解释	当前Miss目标、next Probe、两者阶段与大小。
02	写费恒等式可直接成立；策略需估计未来增长	阶段、当前大小、快照间增量、next INSERT/SWITCH。
03	可行，反馈不完整	自己发出的SPILL、后续Miss、全局Probe时钟。
04	可行，但预测不是实际未来	已知next固定为第一步，后续情景由已观察信息生成。
05	空闲容量证书可行；涉及victim的证书仅局部	阶段、容量、next Probe、候选淘汰计划。

扩展接口单独标记

若主办方允许onAccess通知每次Probe，可实现完整GDS/ARC基线、转移概率和全量影子缓存。原接口不能把这些算法的逐次Hit更新伪装成已经实现。

01：在Miss回调中联合驻留双目标

规则中的机会

onPartitionMiss要求返回SPILL+LOAD动作列表，最后一个动作LOAD当前目标A。题面未明确禁止列表含多个LOAD。因此可在已知next_op=PROBE B、B已处于PROBE阶段时，先加载B，再把加载A作为最后动作。所有动作完成后再执行当前Probe；A+B必须同时容纳。

具体算法

若A=B，普通加载即可。否则把A、B列为保护对象；若s_A+s_B>M，退出联合分支。need=s_A+[B在磁盘时的s_B]。从其他驻留分区选择足够victim，按SPILL V、LOAD B（必要时）、LOAD A的顺序返回。当前A仍计Miss；下一B计Hit。

构造例：两个64字节目标，M=128

方案	行动	I/O	Miss	Score
顺序按需	内存C,D；先换出D加载A，再换出C加载B	2.88	2	22.88
联合驻留	换出C,D；LOAD B；最后LOAD A	2.88	1	12.88

W=100、R=80、penalty=10；例中最终状态都为A,B，I/O字节相同，少一次Miss。该例通过可执行模型检查，不能推出任意序列都改善。

与旧周期预取的区别

该机会来自Miss回调，不必等待每B次操作的周期点。它还保护已经驻留的下一目标，防止为了加载A恰好淘汰马上要用的B。优势来自已知的两步关系和动作顺序，并非依赖完整未来预测。

限制与风险

联合计划可能挤走更有长期价值的分区；后续窗口里不一定更好。若官方把Miss列表限定为唯一LOAD，则需要改为仅保护已驻留的B，预取B仍放周期回调；此分支须在正式驱动下核对。

02：BUILD写入守恒与阶段承诺

可以严格推导的事实

BUILD阶段只允许SPILL，不允许LOAD。一个分区第一次落盘后，后续INSERT直接追加磁盘。令最终Build大小S，落盘时大小s(t)，则：
累计写入字节 = s(t) + [S-s(t)] = S。
假设从空分区开始、确实在BUILD阶段落盘、无压缩或分区拆分，则落盘早晚不改变该分区的完整Build写费S/W。

数值检查

元组大小	第几次插入后落盘	累计写字节	W=100时写费
10,20,30,40	1	100	1.00
相同输入	2	100	1.00
相同输入	3	100	1.00
相同输入	4	100	1.00

由事实产生的策略

对已经判定应在BUILD落盘的候选，不把后续磁盘INSERT的写费又当成额外早落盘惩罚。判断应更多考虑：提前释放当前内存能否保住其他分区、能否避免更多分区也被落盘，以及该分区到PROBE时的保留价值。

“承诺”的含义

采用短暂的阶段决策状态：候选、已决定在BUILD落盘、接近SWITCH。候选进入已决定状态需要可行容量计划与稳定的倾斜增长信号；不是看见大分区就提前写出。当前增长可由大小快照差分得到，结束时刻仍未知，不能声称能识别必然落盘的分区。

必须保留的反例

若分区本可始终留在内存且之后不落盘，其写费为0；提前落盘则要写S字节。因此守恒只比较“同一分区最终都会在BUILD落盘”的时间选择，不能推出“早落盘总是最优”。参考INSERT框架的目标落盘缺陷仍要遵守。

03：带暴露时间的淘汰账本

借鉴来源与新增部分

R2提供保留淘汰历史、利用再次访问反馈的思想。本题新增的是按真实SPILL决策记录暴露区间，并把“很快读回的代价”反馈给所用淘汰规则，而不把接口里看不到的Hit当作0。

记录什么

每次实际SPILL记录(op,pid)、大小、淘汰规则标签、开始Probe时钟 τ 、是否BUILD。 τ 可用hit_count+miss_count计算。若在u时刻因Miss重新加载，记录间隔d=u− τ 和读回费s(u)/R+penalty。主动预取结束该区间时，只记录确认的读费及被避免的需求，不能记为真实Miss。

如何更新保护值

维护已结束/右删失的落盘区间总暴露量E_i和已观察需求数n_i。在选定的局部平稳模型下，可用带先验的(n_i+α)/(E_i+β)作为等待风险信号；更多情况下直接用近距离读回比例和置信区间。选择低置信度时保持温和保护，不把长期驻留的分区断言为冷。

按规则反馈

若某种victim选择频繁在很短的Probe间隔内导致高额读回，增加该规则的风险分。比如规则R连续选中大分区，而这些分区随后立刻Miss，就减少R在相似大小/阶段条件下的使用机会。更新使用可观察的损失，不能称其为“如果采用另一策略就能省下”的已知反事实。

为何强调暴露时间

10次读回/100次磁盘暴露与10次读回/10,000次暴露并不表示相同风险；原版只累计Miss会混淆两者。该账本能表达这一区别，也能处理一直未读回的开放区间。

局限与复杂度

落盘区间本身由策略选择，且Hit不可见，估计有选择偏差，不保证无偏或恢复真实频率。每分区O(1)统计、总空间O(KP)；懒更新避免B=1全表衰减。需实测它是否比简单miss-pressure更好。

04：少量候选计划的多情景比较

单个分区打分会漏掉什么

两个小分区轮流访问时，保住其中一个、反复写出另一个的总成本，取决于整个驻留集合和接下来的动作；把每个分区独立打分未必能表达这一状态演化。本方案比较动作计划，而非只比较单分区标签。

计算流程

先生成4至8个合法计划：原代价策略、最少写字节计划、单个足额分区计划、保护next目标计划、01双目标计划。已知next_op固定为预测窗口第一步；后续4至8步生成少量情景：近距离重复、两分区交替、扫描、热点迁移。情景基于已观察Miss与next样本，不冒充真实未来。

严格模拟题目

在只读影子状态中，逐动作执行SPILL、LOAD、INSERT和Probe计数，不允许重排真正输入、不允许部分加载或忽略预取I/O。模拟LOAD阶段、原子性、每步容量与目标驻留检查。先删除非法计划，再比较预测Score。

计划选择

例如优化 J(V)=平均情景Score(V)+λ×最大情景Score(V)。两项都是时间量纲； λ由独立验证集选择。也可比较相对同一兜底计划的最坏局部差值，防止只追逐一种预测。达到时间预算时直接使用已计算的合法兜底。

离散覆盖的实现

候选集小的时候用保留原字节数的Pareto前沿，或枚举前12个候选的子集；候选集大时采用束搜索与greedy fallback。不能按粗粒度向下取整释放量后宣称安全。复杂度受候选计划数、情景数、窗口长度及模拟victim生成费用控制。

创新定位与限制

短窗口情景规划是通用优化思想；此处原创设计是针对原子分区、双目标回调、写费守恒与观测受限信号构造的计划生成器。它不提供全局最优或竞争比证明，且需最大规模的计算预算实验。

05：预取证书与等待信息价值

一级证据：空闲容量的确定前缀收益

周期回调已知下一条是Probe A，A在磁盘、已处于PROBE，空闲容量≥s_A。现在LOAD A与等到下一Probe的Miss才LOAD A，读费相同、下一Probe后物理驻留状态相同；提前加载让Miss减少1，前缀Score严格少penalty。证书只保证这个前缀；自适应控制器若使用真实Hit/Miss计数，后缀决策可能不同。

二级证据：比较相同victim计划

若等待策略也会选择完全相同的victim集合，则预取只是把同一组SPILL/LOAD提前，不能把这些本来也要付出的写费全算作预取的额外成本。上一版victim_cost<penalty是保守启发式，不是精确的增量收益判据。

为什么仍不能总是预取

周期回调只知道下一条A；等到A的Miss回调，State.next_op已经变成再下一条操作，可能提供新的victim选择信息。提前落盘会放弃这条信息。若不知道接下来会访问C，预取可能误淘汰C；等待则能保护C。

三级证据：考虑信息损失

对若干可能的再下一Probe U，比较“现在预取的计划”与“等到A时能看到U的需求计划”。只有在所有所测情景的局部差值≤0时，给予覆盖该有限情景集的证书；未覆盖输入及更长后缀仍无保证。不能把一个预测情景的正收益叫作确定收益。

构造反例

M=128；A、C、D各64，初始C,D驻留。下一条A，再下一条C。提前误写C后，A命中但C未命中；等待A的Miss时看见next=C，转而写D。W=100、R=80、penalty=10时，提前方案Score=12.88，等待方案=11.44。

原创贡献

把“预取收益”和“等一拍获得的信息”放进同一判断，区别确定前缀结论、有限情景证据与经验预测。该反例已用模型验证，说明无条件淘汰式预取并非普遍优越。

合法性、边界与失败模式

情况	处理
A与B总大小>M	O1联合驻留不可行，不能以部分LOAD或重排Probe规避；退回普通需求加载。
下一目标处于BUILD	禁止预加载；只考虑合法SPILL与当前目标加载。
下一目标已驻留	将其设为保护候选，比较保住它和不保护的计划；不重复LOAD。
下一目标与当前相同	一次需求加载已经能满足连续访问，不多做I/O。
分区大小超过M	必须完整加载的Probe无解。不能用原创策略掩盖题面可行性缺口。
INSERT回调选中当前目标	参考引擎账目缺陷仍在；兼容实现继续排除当前目标。周期预先SPILL不等同于当前回调SPILL。
虚拟试运行	只复制状态评估动作，不改变真实引擎、不产生虚构命中；最终动作列表仍由引擎逐个执行。
样本稀少或热点变化	O3的不确定性提高；O4保留扫描/迁移情景；回到代价基线，不能以无数据推断冷。

前缀结论不能扩大成全局保证

相邻两步减少一次Miss，可能付出将来冷分区重载的代价。把每个模块的改善窗口、比较对象、模型假设与观察限制写清楚，是新方案可信度的一部分。

已完成的构造检查与证据等级

检查	结果	证据范围
01相邻双目标	相同I/O 2.88；Miss由2变1；Score由22.88变12.88。	给定合法初始状态的两步例；每动作容量验证。
02写费守恒	4种落盘时点均写100字节，写费1.00。	固定4元组示例，并有恒等式推导。
05空闲预取	读取均0.80；Score由10.80变0.80。	明确下一Probe且无需victim的前缀。
05信息反例	提前错误淘汰Score=12.88；等待信息Score=11.44。	证明淘汰式预取存在退化序列。
观测不可还原	两段不同Hit历史有相同计数及驻留快照。	解释完整访问预测为何需要额外接口。

如何复现

运行 python design/innovation_checks.py，结果写入innovation_checks.json。验证程序不需要第三方依赖；它是小型构造检查，不是提交用Hash Join执行器。

尚未证明的内容

没有03/04的性能曲线，没有五方案组合的全量实现，没有最大规模或官方评测成绩。旧版200组数据只对应旧CostAwareScheduler模型；本报告没有把旧成绩归到新增方案名下。

对照必须匹配

每组使用相同输入、同一合法性修复、相同等分区tie-break、相同评分与I/O计费。01检查必须将B作为独立变量，避免把周期频率变化的影响误当作联合驻留收益。

建议主线与消融实验

建议主线

以O1双目标联合驻留为可演示功能，O2写费守恒为理论解释，O5信息价值为稳定性边界；用O3修正观测偏差，再按计算预算引入O4。这样有规则推导、合法动作、具体数值和退化反例，适合答辩。

消融	固定条件	要回答的问题
旧基线 vs +O1	B=1/10/1000，W/R与penalty相同	新回调机会是否在大B时也减少Miss？
原BUILD权重 vs O2阶段决策	最终大小及切换位置相同	早释放能否减少其他分区写费？误判会付出多大代价？
miss-pressure vs +O3	均匀、倾斜及热点迁移	暴露时间反馈是否减少很快读回？低样本是否退化？
密度计划 vs +O4	双分区循环、多算子交错、扫描	集合状态评估收益是否超过计算成本？
阈值预取 vs +O5	信息反例、空闲/满内存	证书机制能否保住确定收益，并降低错误淘汰？

报告指标

每组Score、Hit/Miss、读写字节、动作数量、每次动作峰值内存、CPU墙钟及规划耗时P95；按题面逐组BASE/VAL累加成绩。参数在验证集调整，最终测试集不参与调参。

落地先后

先核对多LOAD解释并在官方C++框架跑O1与O5；其次实现O2日志和O3账本；最后增加O4。任何一步都保留已有合法需求加载策略作为兜底，不把复杂预测放在容量安全路径上。

参考文献与原创归属

R1

Cao, Pei; Irani, Sandy. Cost-Aware WWW Proxy Caching Algorithms. USITS, 1997.

打开论文原文

R2

Megiddo, Nimrod; Modha, Dharmendra S. ARC: A Self-Tuning, Low Overhead Replacement Cache. FAST, 2003.

打开论文原文

R3

Kuiper, Laurens; Groß, Paul; Boncz, Peter; Mühleisen, Hannes. Saving Private Hash Join. PVLDB 18(8):2748-2760, 2025. DOI:10.14778/3742728.3742762.

打开论文原文

R4

Lykouris, Thodoris; Vassilvitskii, Sergei. Competitive Caching with Machine Learned Advice. ICML, PMLR 80:3296-3305, 2018.

打开论文原文

来源与推导的边界

R1支持代价与大小的缓存思想；R2支持历史淘汰反馈；R3支持外部Hash Join和全局内存管理的相关性；R4支持预测需要兜底这一研究路线。它们不是本报告五个具体方案的效果证明。

O1动作重排、O2守恒等式、O5信息反例与数值由题面规则独立推导；O3是在历史淘汰思想上加入暴露区间、读回损失与规则反馈的具体设计；O4是把本题候选动作集合放进有预算的情景试运行。

书目查验日期：2026年10月13日。作者、年份、标题与原始论文首页核对；没有引用搜索结果日期作为论文发表时间。